

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..28

TROUBLESHOOTING

 Copy as Markdown

Troubleshoot Microsoft sign-in

This page helps you identify common Sign in with Microsoft failures, explains why they occur, and shows how to resolve them. For underlying issues with the OAuth callback library, see the [flutter_web_auth_2 documentation](#).

Setup checklist

Go through this before investigating a specific error. Most problems come from a missed step.

Microsoft Entra ID portal

- ☐ Created an app registration in [Microsoft Entra ID](#).
- ☐ Chose **Supported account types** that cover everyone who should sign in, and that match your `tenant` setting.
- ☐ Registered every redirect URI you actually use under **Authentication**, each under the right platform: **Web** for `https://your-domain.com/auth.html`, **iOS / macOS** with your bundle ID, and **Android** with your package name and signature hash.
- ☐ Created a client secret and copied its **Value** (not the Secret ID). Microsoft only shows it once.

Server

- ☐ Added `microsoftClientId` and `microsoftClientSecret` to `config/passwords.yaml` under the matching environment (`development:` for local, `production:` for prod), or set the matching `SERVERPOD_PASSWORD_microsoftClientId` and `SERVERPOD_PASSWORD_microsoftClientSecret` environment variables. The `microsoftTenant` key is optional and defaults to `common`.
- ☐ Added `MicrosoftIdpConfigFromPasswords()` to `identityProviderBuilders` in `server.dart`.
- ☐ Created an endpoint that extends `MicrosoftIdpBaseEndpoint`.



- ☐ Started the server with `serverpod start`, then created and applied the migration (pressed **M**).

Flutter app

- ☐ Added `client.auth.initializeMicrosoftSignIn(clientId: ..., redirectUri: ...)` after `client.auth.initialize()` inside `initializeClient()` in your Flutter app's `lib/client.dart`.
- ☐ Both `clientId` and `redirectUri` match values registered on the app registration.
- ☐ The `tenant` passed on initialization matches the server's `tenant` setting. Both default to `common`.
- ☐ On **Android**, added the `flutter_web_auth_2` `CallbackActivity` to `AndroidManifest.xml` with the **exact** scheme and host used in your callback URL.
- ☐ On **Web**, created `web/auth.html` in your Flutter project with the callback script from [Web callback page](#) (`auth.html`).
- ☐ On **web**, ran Flutter on a fixed `--web-port` matching the port in the registered redirect URI.

Sign-in fails with a redirect URI error

Problem: The browser opens the Microsoft sign-in page, but instead of completing, Microsoft shows an error page saying the redirect URI is not valid for the application. The app never receives a callback.

Cause: The `redirectUri` your Flutter app sent to Microsoft does not exactly match any redirect URI registered on your app registration, or the URI is registered under the wrong platform.

Resolution: Open your app registration's **Authentication** page and verify the registered redirect URIs match your app's `redirectUri` exactly. The match is strict: scheme, host, port, path, casing, and trailing slashes all count.

Common mistakes:

- Trailing slashes (`https://your-domain.com/auth.html/`) or port differences.
- Wrong scheme (`http` vs `https`, or a mismatched custom scheme like `myapp:` vs `MyApp:`).
- The URI registered under the wrong platform, for example a web URL added under a native platform.
- Flutter dev server running on a random port. Pass `--web-port=<port>` to `flutter run` so the origin is stable across restarts.

Callback never returns to the Flutter app

Problem: The user signs in on the Microsoft page successfully, but the Flutter app never receives the result. The browser sits on a blank page or the sign-in window hangs.

Cause: The browser was redirected to a URL that does not serve the callback page (web), or the callback custom scheme is not registered with the platform (mobile).

Resolution:

- **Web:** Confirm `web/auth.html` exists in your Flutter project and contains the callback script from [Web callback page \(auth.html\)](#). The page posts the result back to its own origin, so your Flutter web app must be served from the same scheme, host, and port as the callback URL.
- **Android:** Verify the `<data android:scheme="..." android:host="..." />` values in `AndroidManifest.xml` match the scheme and host in your callback URL exactly.
- **iOS / macOS:** Universal Links require HTTPS callback URLs and associated-domain entitlements. Standard custom-scheme callbacks work without extra configuration.

Sign-in fails with an access token verification error

Problem: The sign-in flow completes on the Microsoft page, but the app then reports "An error occurred while verifying the Microsoft access token. Please check your Microsoft account and try again. If the problem persists, please contact support."

Cause: The server threw a `MicrosoftAccessTokenVerificationException`. This exception is deliberately generic so it does not leak details to potential attackers. It covers every server-side failure between receiving the authorization code and validating the account:

- The token exchange with Microsoft was rejected. Typical reasons are a wrong or expired client secret (Microsoft Entra ID secrets always have an expiry date), a `tenant` mismatch between the app and the server, or a reused or expired authorization code.
- The user info request to Microsoft Graph failed, usually because custom scopes dropped `https://graph.microsoft.com/User.Read`. See [Sign-in breaks after changing the scopes parameter](#).
- A custom `microsoftAccountDetailsValidation` callback threw. See [Custom account validation](#).
- A `getExtraMicrosoftInfoCallback` threw. See [Calls from getExtraMicrosoftInfoCallback fail or block sign-in](#).

Resolution: The server logs the underlying cause at debug level for token exchange, user info, and callback failures. A throwing validation callback surfaces only as a generic invalid-user-info error. Check the server logs with debug logging visible, then work through the matching cause above. If the log shows a token exchange error, verify the client secret is current and that the `tenant` values on the server and in `initializeMicrosoftSignIn` are the same.

clientId or redirectUri missing at initialization

Problem: The app throws an `ArgumentError` on startup saying the Microsoft client ID or redirect URI is required.

Cause: Microsoft has no native platform-specific config files. The `clientId` and `redirectUri` must be passed explicitly to `initializeMicrosoftSignIn`, or read from `--dart-define`.

Resolution: Either pass the values directly:

```
await client.auth.initializeMicrosoftSignIn(  
  clientId: 'your-microsoft-client-id',  
  redirectUri: 'myapp://auth',  
);
```

Or read them from `--dart-define`:

```
flutter run \  
  --dart-define=MICROSOFT_CLIENT_ID=your-microsoft-client-id \  
  --dart-define=MICROSOFT_REDIRECT_URI=myapp://auth
```

The tenant has no environment variable. Pass it as an argument when you initialize Microsoft sign-in. See [Configuring client IDs on the app](#).

The sign-in button does nothing and `onError` never fires

Problem: Tapping the Microsoft sign-in button appears to do nothing, or the flow opens and closes, and the `onError` callback is never called.

Cause: The `onError` callback only receives errors that are safe to show to the user. Configuration mistakes and flow interruptions are not passed to it:

- A `StateError` because `initializeMicrosoftSignIn` was never called before the button was used.
- The user cancelled the sign-in window.
- The OAuth flow failed before reaching the server, for example when Microsoft returned an error on the callback.

Resolution: Check the debug console. The controller prints every failure as `[MicrosoftAuthController] Authentication error: ...` before deciding whether to surface it. If the log shows a `StateError`, move `initializeMicrosoftSignIn` so it runs during app startup, before any sign-in UI is shown.

Changed initialization values do not take effect

Problem: You changed the `clientId`, `redirectUri`, or `tenant` passed to `initializeMicrosoftSignIn`, but the app keeps using the old values.

Cause: Initialization is idempotent. Only the first call in the app's lifetime stores the configuration, and later calls return without changing it. A hot reload keeps the old configuration alive.

Resolution: After changing any initialization value, do a hot restart or fully restart the app. Either one re-runs initialization with the new values.

Sign-in breaks after changing the scopes parameter

Problem: Sign-in worked, then you set the `scopes` parameter on `MicrosoftSignInWidget` or `MicrosoftAuthController` to request extra permissions, and now every sign-in fails with the access token verification error.

Cause: The `scopes` parameter replaces the default scopes instead of adding to them. The server fetches the user's details from Microsoft Graph during sign-in, which requires the `https://graph.microsoft.com/User.Read` scope. Dropping it breaks the account details fetch, and dropping the OpenID Connect scopes breaks the sign-in itself.

Resolution: Include the defaults alongside your extra scopes:

```
MicrosoftSignInWidget(  
  client: client,  
  scopes: [  
    ...MicrosoftAuthController.defaultScopes,  
    'https://graph.microsoft.com/Calendars.Read',  
  ],  
)
```

See [Requesting additional Microsoft scopes](#) for the default scope list.

Users see a "Need admin approval" screen

Problem: A user signs in and Microsoft shows a screen saying the app needs admin approval instead of completing the flow.

Cause: One of the requested scopes requires admin consent in the user's organization. Work and school tenants can require an administrator to approve permissions before any user can grant them.

Resolution: Ask an administrator of that tenant to grant consent for the app's permissions, under **API permissions** in the app registration. Alternatively, remove the scope that triggers the requirement if you do not strictly need it. The default scopes normally do not require admin consent.

Some account types cannot sign in

Problem: Sign-in works for some users, but others get a Microsoft error page saying their account cannot be used with this application.

Cause: Two settings restrict which accounts can sign in, and both must allow the user:

- The **Supported account types** choice on the app registration.
- The `tenant` value in your configuration. Use `common` for personal and work/school accounts, `organizations` for work/school accounts only, `consumers` for personal accounts only, or a tenant ID for a single organization.

Resolution: Align both settings with your audience. Set the `tenant` in the server configuration and pass the same value to `initializeMicrosoftSignIn` in the Flutter app. Update **Supported account types** on the app registration to match. See [Tenant configuration](#).

Sign-in works on mobile but fails on web, or the reverse

Problem: Microsoft sign-in works on Android and iOS, but fails on web with the access token verification error, or the other way around.

Cause: Microsoft requires the client secret during token exchange for web apps and rejects it for native apps. Serverpod handles this through the `isWebPlatform` flag on the login endpoint, which the provided widgets pass automatically. A platform-specific failure usually means the redirect URI is registered under the wrong platform on the app registration, so Microsoft applies the wrong rules to the exchange.

Resolution: On the app registration's **Authentication** page, confirm web redirect URIs sit under the **Web** platform and native ones under the **iOS / macOS** or **Android** platforms. If you built a custom flow that calls the login endpoint directly, pass `isWebPlatform: true` on web and `false` everywhere else.

Sign-in succeeds but the user has no email, or the wrong one

Problem: The user signs in successfully, but the server-side `MicrosoftAccountDetails.email` value is `null`, or the stored email is not an address the user recognizes.

Cause: Microsoft Graph does not always return a mail address. The provider reads the `mail` field first and falls back to `userPrincipalName`, then stores the result in lowercase. The `mail` field can be empty for accounts without a mailbox, and `userPrincipalName` is a sign-in identifier that is not always a real address, especially for guest accounts. A custom validator that requires an email will block these users, and the app then shows the access token verification error.

Resolution:

- If you do not strictly need an email, relax your validator. The default validator only checks that `userIdentifier` is non-empty. See [Custom account validation](#).
- Treat the stored email as informational rather than as a verified mailbox, or collect an email in your own onboarding flow when you need a reliable one.

Users have no profile photo

Problem: Users sign in successfully, but their profiles have no photo.

Cause: The provider fetches the photo from Microsoft Graph when `fetchProfilePhoto` on `MicrosoftIdpConfig` is enabled, which is the default. A failed photo fetch never fails the sign-in. It is logged and skipped. Microsoft also does not return a photo for every account. For returning users, the photo is only set when the profile does not already have one.

Resolution: Confirm `fetchProfilePhoto` is not set to `false` if you expect photos. Accept that some accounts have none. If sign-in speed matters more than photos, disable the option. See [Profile photos](#) for how photos are stored.

Calls from `getExtraMicrosoftInfoCallback` fail or block sign-in

Problem: After adding a `getExtraMicrosoftInfoCallback`, sign-ins start failing with the access token verification error, or external API calls inside the callback fail intermittently.

Cause: The callback runs on **every** authentication attempt, before the system determines whether the user is new or returning. Any exception it throws aborts the whole sign-in.

Resolution:

- Wrap external calls in `try/catch` so a Microsoft Graph outage or rate limit does not block sign-in.
- Cache what you fetch in your own tables, keyed by `MicrosoftAccountDetails.userIdentifier`, instead of re-fetching on every sign-in.
- Do not create `MicrosoftAccount`, `UserProfile`, or `AuthUser` records inside the callback. That breaks new account detection and profile creation. See [Accessing Microsoft APIs on the server](#).

Sign-in works in dev but fails after deploy

Problem: Microsoft sign-in works locally but fails in production with a redirect URI error or the access token verification error.

Cause: The production redirect URI is not registered on the app registration, or the production Flutter build is using the dev `redirectUri`, or the production server has no Microsoft credentials configured.

Resolution:

1. Confirm the production redirect URI is registered on the app registration alongside the development one. Both should remain registered so dev and prod work simultaneously.
2. Confirm your production Flutter build is initialized with the production `redirectUri`. The simplest way is to read it from `--dart-define` and pass the production value in your CI/CD or `flutter build` step. See [Using environment variables](#).
3. Confirm the production environment provides `microsoftClientId` and `microsoftClientSecret`, in `config/passwords.yaml` under `production:` or through the `SERVERPOD_PASSWORD_` environment variables.

Server fails to start with a missing password error

Problem: The server crashes on startup with a `PasswordNotFoundException` naming `microsoftClientId` or `microsoftClientSecret`.

Cause: You use `MicrosoftIdpConfigFromPasswords()`, and the named key is missing from `config/passwords.yaml` and from the environment. The exception message names the exact key and the environment variable it also looked for.

Resolution: Confirm both keys exist under the section matching your run mode (`development:` when running locally with `serverpod start`, `production:` when deployed):

```
development:
  microsoftClientId: 'your-microsoft-client-id'
  microsoftClientSecret: 'your-microsoft-client-secret'
  microsoftTenant: 'common' # optional, defaults to common
```

Quoting the values is a safeguard. YAML parses unquoted values that look like numbers or booleans as those types instead of strings.

Server crashes on first Microsoft sign-in with "no such table"

Problem: The server builds and starts, but crashes when a user tries to sign in with Microsoft. The error cites a missing table such as `serverpod_auth_idp_microsoft_account`.

Cause: The database migration that creates the provider's tables was never created or applied.

Resolution: In the running `serverpod start` terminal, press **M** to create and apply the migration.

Android sign-in opens Microsoft but the callback never fires

Problem: On Android, tapping the Microsoft sign-in button opens the Microsoft authorization page in a browser, but after authorizing, the browser stays open and the Flutter app never resumes.

Cause: The `CallbackActivity` in `AndroidManifest.xml` is missing, has a wrong scheme or host, or `android:exported` is not set to `true`.

Resolution: Open `android/app/src/main/AndroidManifest.xml` and confirm the `CallbackActivity` block exists with `android:exported="true"` and the `<data android:scheme="..." android:host="..." />` values matching your callback URL exactly. The block is shown in [Android setup](#).

After editing the manifest, run `flutter clean` and rebuild.

Related

- [Setup](#): set up Microsoft sign-in on the server and in your app.
- [Customizations](#): configuration options and sign-in UI customization.
- [UI components](#): the sign-in widgets and how to compose them.